```
clc
clear
close all

s = tf('s');

% Options for Bode-Plots
bodeopts = bodeoptions;
bodeopts.FreqUnits = 'Hz';
bodeopts.MagUnits = 'dB';
bodeopts.Grid = 'on';
bodeopts.PhaseWrapping = 'on';
bodeopts.YLimMode = 'auto';
bodeopts.XLimMode = 'auto';

% Options for Step-Plots
stepopts = timeoptions;
stepopts.Grid = 'on';
stepopts.YLimMode = 'auto';
stepopts.XLimMode = 'auto';

%% =Parameters=======================================

%-Controller-Type----------------------------------
% 0: P
% 1: I
% 2: PI
% 3: PD
% 4: PID
% 5: PID-T1
% 6: PID-T2
controller_type = 4;

%-Kp,-Ki,-Kd-values--------------------------------
Kp = 600;
Ki = 10;
Kd = 100;

% Filterfactor (-> lowpass filter 1. ord.)
% higher N -> closer to ideal
% lower N -> stronger filtering
N = 1000;

%-Plant-Type---------------------------------------
% 0: no Plant
% 1: Integrator
% 2: PT1
% 3: PT2
% 4: Totzeitglied
% 5: Combination: Integrator + PT1
plant_type = 0;
```

```
%% -Controller-----------------------------------------

switch controller_type
    case 0 % P
        C = tf(Kp, 1);
        controller_name = 'P';
    case 1 % I
        C = Ki/s;
        controller_name = 'I';
    case 2 % PI
        C = Kp + Ki/s;
        controller_name = 'PI';
    case 3 % PD
        C = Kp + Kd*s;
        controller_name = 'PD';
    case 4 % PID (ideal)
        C = Kp + Ki/s + Kd*s;
        controller_name = 'PID';
    case 5 % PID-T1 (D-Anteil mit TP1)
        Tf = Kd/N;
        C = Kp + Ki/s + (Kd*s)/(Tf*s + 1);
        controller_name = 'PID-T1';
    case 6 % PID-T2 (D-Anteil mit TP2)
        C = Kp + Ki/s + (Kd*s)/(1 + (Kd/N)*s + (Kd/N)^2 * s^2);
        controller_name = 'PID-T2';
    otherwise
        error('invalid controller type')
end

%% -Plant----------------------------------------------

switch plant_type
    case 0 % Plant off
        P = 1;
    case 1 % Integrator
        P = 1/s;
        plant_name = 'Integrator';
    case 2 % PT1
        T1 = 0.5;
        Kp_strecke = 1;
        P = Kp_strecke / (1 + T1*s);
        plant_name = 'PT1';
    case 3 % PT2
        wn = 10;
        zeta = 0.5;
        P = wn^2 / (s^2 + 2*zeta*wn*s + wn^2);
        plant_name = 'PT2';
    case 4 % dead-time element (Pade-Approximation)
        Tt = 1;
        P = pade(exp(-Tt*s), 4);
        plant_name = 'dead-time element';
    case 5 % Integrator + PT1
        T1 = 0.5;
        P = 1 / (s*(1 + T1*s));
```

```
    otherwise
        error('invalid plant-type')
end

%% -Loop-calculations--------------------------------

L = C * P;            % Open Loop
T = L / (1 + L);      % Closed Loop without prefilter

%% -Plots-----------------------------------------

% Frequency Response Controller
figure(1)
bode(C, bodeopts, 'r')
title(sprintf('Frequency Response \n%s-Controller', controller_name))

if controller_type ~=4 & controller_type ~=3
    % Step Response
    figure(4)
    step(C, stepopts, 'b')
    title(sprintf('Step Response \n%s-Controller', controller_name))
    % draw Line on Y-axis from 0 to start of curve
    [y, t] = step(C);
    hold on
    plot([0 0], [0 y(1)])
    hold off
end

if plant_type ~=0
    % Frequency Response Plant & Open Loop
    figure(2)
    bode(P, bodeopts), hold on
    bode(L, bodeopts)
    legend(sprintf('Plant: %s', plant_name), 'Open Loop L')
    title('Frequency Responses')

    % Nyquist-Diagramm
    figure(3)
    nyquist(L), grid on
    title(sprintf('Nyquist-Diagram Open Loop \n%s-Controller', controller_name))

    % Step Response
    figure(5)
    step(L)
    title(sprintf('Step Response Closed Loop \n%s-Controller', controller_name))
end
```